Order Book and Matching Engine

Trent Houle

May 2026

Motivation and Problem Statement

I wanted to create an order book and matching engine so that I could better understand the technology behind a tool that I interact with frequently. I also wanted to work on something for my final project that would be both complicated yet feasible, as well as something I can iteratively improve on and make faster.

How to Run

Running `make` will compile:

1. `runSampleData.cpp`
2. `testOrderBook.cpp`
3. `benchmark.cpp`

Running `make clean` will clean up the compiled binaries.

runSampleData.cpp

This file processes the sample LOBSTER data and prints out the state of the resulting order book, i.e. the unexecuted bids/asks.

testOrderBook.cpp

This file runs the order book tests.

benchmark.cpp

This file runs the benchmarking code, which is responsible for generating a JSON file that `visualize_bench.py` can parse and then generate multiple corresponding graphs showcasing the benchmark performance.

System Design and Architecture

We need objects to represent orders (bid and ask), the order book, and the order history.

I am currently planning to only tackle "good till canceled" orders, although I may expand on this in the future with other types such as "fill or kill". I also am only dealing with integer shares at the

moment, but the orders are templatized so handling fractional shares should be an obvious next step.

We start with an order book. The order book has a `min_heap` for ask orders and a `max_heap` for bid orders. This allows us to keep track of the lowest prices users are willing to sell at, as well as the highest prices users are willing to pay. We also keep an `unordered_map` connecting order IDs to orders.

When an order is created it initially has no ID or timestamp. These only become properly initialized (ID is initialized with a sentinel value for security) when the order is added to the order book.

When an order is added to the order book, an ID and timestamp are added to it. The ID is monotonically increasing and represents the order in which orders are added to the book. We can use this to compare orders for priority, with earlier orders taking precedence.

After initializing ID and timestamp, the order book checks to see if it is possible to match the new order with an existing order. If it is possible to match the order then we call `matchOrder()`, and get back a vector of trades. If there is any remaining quantity to the current order, it is then added to the book. If the order cannot match, it is simply added to the correct side.

I have read online about different approaches to data structures for order storage. At the moment I am using vectors as heaps, but I may change this in the future.

The orders are stored as a `shared_ptr` in both the heaps and the `orderList`, allowing us to access arbitrary orders in $O(1)$ time from the `orderList` to make changes to them, such as cancellation or modification. This is what allows the lazy deletion strategy, mentioned later on, to work.

When handling modifying orders, instead of modifying the original order in place and requiring a heap correction, I just cancel the original order and create a new one with the desired modifications. This strategy also creates a new ID and bumps the newly created order to the back of the line in terms of priority, which was a purposeful design choice—changed orders should lose their place in line, or the system could be gamed by keeping an incredibly high or low priced order and then changing it to the desired price to instantly execute a trade. I require both arguments for modification (price and quantity) even if only one is modified, because price and quantity share the same underlying type, making it impossible to distinguish between overloads for each as I had originally planned.

I decided to use a lazy deletion strategy. When an order is modified, filled, or canceled, it is not immediately removed from the `orderList` and bid/ask heaps but instead marked as inactive. Then when orders are processed in the matching stage, we check to see if a matching order is active, and if it is not we handle removing it from the data structures at that point. This keeps the removal cost low as we are only ever popping them from the top of the heap, which is $O(\log N)$ instead of the $O(N)$ cost for deletion at an arbitrary location in the heap.

The vector heap logic needs a comparator function, so I had to create one for both bids and asks. Because the logic is similar but different for each of them, I templatized it on the `Side` class, allowing the compiler to create versions for each type at compile time. When sorting for the view we want to be able to sort in the opposite direction, as sorting and heap logic compare in the opposite direction. To handle this I used a lambda function to flip the arguments and have it compare with the same logic, but in the opposite direction.

I wanted to create formatters for orders, and had plans for more, but after finishing the order formatter it would not compile in C++20. I looked around online and this seems to be a result of the limitations of `libc++` in Apple compilers. I left the commented out code in there just to show

that I attempted it and was met with the unfortunate reality that Apple doesn't want us to have fun.

Matching Algorithm

I am using a price-time priority algorithm. This means that orders will be prioritized by price first, and then order time (represented by ID). For example: if a sell order comes in that can match, and two buy orders are both at the lowest price and can match, the buy order with the smaller ID will be used.

TODO

This was created after I had made significant progress in the orders, order book, trades, and matching engine. It is not a complete list of tasks.

- Cancel order function
- Match order function
- Can match function
- Fix comparator for bid/ask heaps
- Finish trades struct
- Get next order function
- Add `modifyOrder()`
- Change `addOrder()` to take in args and construct the order
- Fix `main.cpp` to use the new `addOrder()` syntax
- Level 1 data available
- Level 2 data available
- Get testing data
- Implement data pipeline
- Write unit tests
- Do testing
- Benchmark/profiling and project write-up
- Level 3 data available (maybe not, we will see)
- Add Makefile
- Create `exchange.cpp`
- `printOrderHistory()` and `printTradeHistory()`
- Support for different order types

Evolution of the Design

There are many fully built examples of order books and matching engines online, but I would like to approach it without using them as a framework. I will look to examples if I get stuck, but that means I will probably be making some mistakes and inefficient design choices.

1. Added timestamp to represent when an order was added.
2. Changed from using a list to using vectors to represent the order book sides. I decided to use vectors and heapify them.
3. Changed ID to be created when an order was added, instead of an argument to the order constructor.
4. Changed ID to be the main comparison of time instead of timestamp, because timestamp was finicky.
5. I was originally using a spaceship operator overload for comparison between orders, but I switched to a templatized struct comparator for bids and asks because this was not working for the `min_heap` comparisons.
6. I'm trying to decide if `matchOrder()` should use cancel order or have its own logic for removing the top order. I think it is fundamentally different from `cancelOrder()` so I am leaning towards using its own logic.
7. I decided to keep the logic separate, and `matchOrder()` has its own remove. I also read online and found a Stack Overflow post where someone recommended keeping a set of deleted elements, and if you try to access the top element in the heap and it has previously been deleted, you remove it and keep going. This avoids having to remove arbitrary elements from the heaps. (Stack Overflow post linked as reference #7.)
8. Created a `getNextOrder()` function that is responsible for getting the top order and handling deleted orders in the heap.
9. Added status field to `Order`, initialized to 1. Added `cancelOrder()` member method which sets status to 0, as well as `isActive()` which returns status. This allows us to check if orders are active or canceled. When an order is canceled we just set active to 0, and then if we ever run into that order again it is removed from the system at that time.
10. Changed `getNextOrder()` to return `optional<reference_wrapper<Order<T>>>` so that if there is no correct next value that information propagates up instead of throwing a logic error like it did before.
11. Removed `canMatch()` because it is now redundant; `getNextOrder()` does the error checking and we can just check for a price match in `matchOrder()`.
12. Added `tradeHistory` member variable to `OrderBook`, so that we have a complete list of trades.
13. Changed `addOrder()` return type from `Trades` to `Id`; now it returns the ID of the added order so users can get that info and then modify or cancel orders. Trades completed in `addOrder()` are added to `tradeHistory`.
14. As I was working on `modifyOrder()`, I decided the best way to handle this was to just cancel the existing order and create a new one. This avoids a lot of the complex logic related to

modifying the heap to deal with a price change, matching the order if it can now match, etc.

15. Finished `modifyOrder()`. Now I'm thinking I should change `addOrder()` to only require the args to construct an order, rather than take in a constructed order. I might make a helper function that is exposed to users, which creates an order with their order info and then passes a constructed order to `addOrder()`.
16. Trying to decide what behavior `modifyOrder()` should exhibit when modifying a canceled order. I am going with "throw a logic error" because you shouldn't be able to modify a canceled order, but it would also make sense to just let nothing happen.
17. Added a Makefile to improve ease of compiling. I've only been working with a `main.cpp` file, but soon I will have other files so this is preventative.
18. Wanted to add a formatter for `Order`, but due to `libc++` limitations this was not working. I instead created an `operator«` overload for orders and trades.
19. Added `OrderBookIterator` to assist with providing level data.
20. After creating the order book iterator, I added the `OrderBookView` class as well as `buildOBView()` to handle the creation of an order book view for either side. This allows us to get the bids/asks as a sorted vector, copied from the heap. This also gives us easy access to level 1 data as we can see the top bid/ask by creating an `OrderBookView`.
21. Changed print methods to use `OrderBookView`.
22. Separated the large `orderBook.h` file into `types.h`, `order.h`, and `orderBook.h` for clarity.
23. Finished `runSampleData.cpp`, which reads in from a sample data file and replays the trades, modifications, and cancellations as given in the sample data. It is not completely correct as there are features I cannot replicate (such as hidden limit orders that are not included in the sample data and only show up in completed trades). It's a good start, and I plan to build on this.
24. I have been doing manual testing throughout, with a simple test file, but I went ahead and created a much larger testing file `testOrderBook.cpp` which covers all the main tests an order book should have.
25. Added `benchmark.cpp` to handle running the benchmark and outputting a JSON file with benchmark info, as well as `visualize_bench.py` which creates visualizations of the benchmarks for simpler comprehension.

Performance and Testing

Testing is handled in `testOrderBook.cpp`. The tests cover the core behaviors the order book needs to operate correctly: adding orders, matching at the correct price-time priority, partial fills, cancellations, modifications, and edge cases like attempting to modify a canceled order. All tests are passing, and I have been using them to verify correctness through my refactoring.

Benchmarking is handled in `benchmark.cpp` and `visualize_bench.py`. The benchmark replays the LOBSTER AAPL sample data (83,255 events per run) through the order book 100 times, with 10 warmup runs discarded beforehand. To prevent the CPU branch predictor from learning the exact pattern of the dataset across runs, each run receives a freshly "jittered" copy of the data where every add-order price is independently randomized by $\pm 2\%$. This means each run sees a

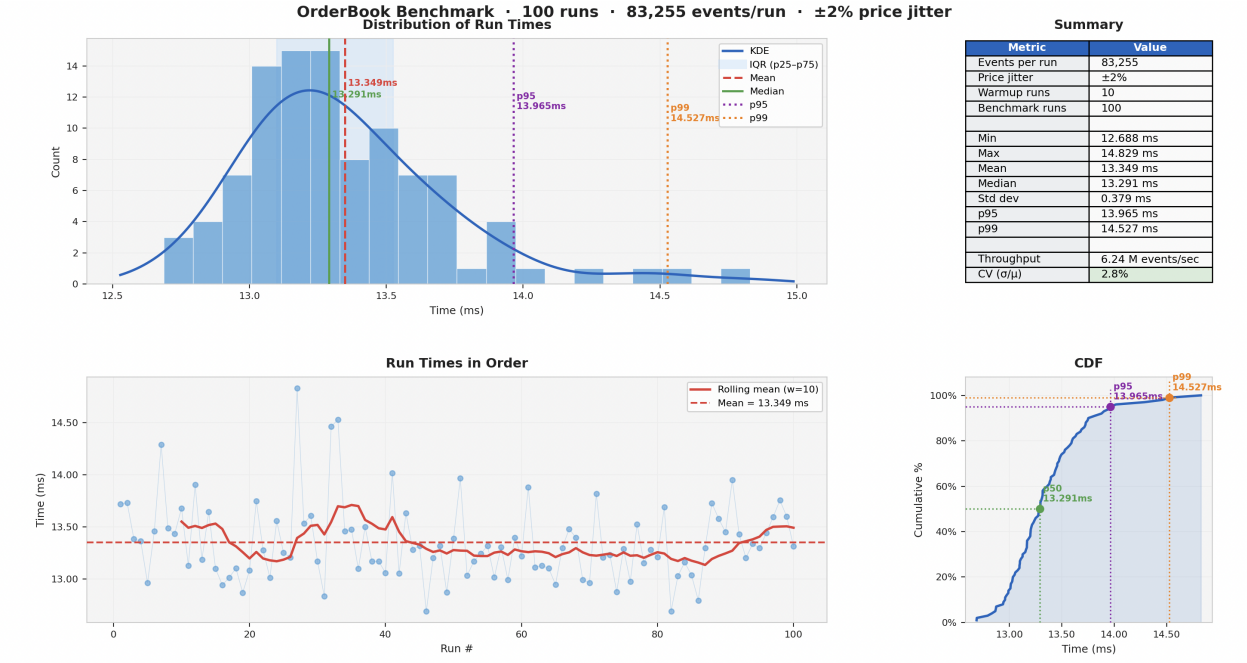

Figure 1: Order book benchmark results: 100 runs, 83,255 events/run, ±2% price jitter.

different mix of trades, which is a practical stand-in for using a different dataset with each test. This is actually one of the main questions I had for Matt Godbolt—"Do you have any advice on how to use testing data to prevent the branch predictor from catching on with repeated testing?"—but I didn't get the chance to ask him during class.

Currently I am seeing mean throughput at 6.24 million events per second, with a mean run time of 13.349 ms and a median of 13.291 ms. The mean and median being so close means we are not seeing significant outliers. The standard deviation is 0.379 ms, indicating fairly stable and consistent measurements.

The p95 and p99 latencies are 13.965 ms and 14.527 ms respectively, and at only 9% above the median this is a good sign, as it is important for an order book that even in the worst cases you do not see significant latency.

Looking at the run-order plot, there is no clear downward trend early on and no sustained upward drift. The occasional spikes are likely OS scheduling noise, which is expected.

I haven't done any serious profiling yet to identify where time is actually being spent. The heap operations and the `unordered_map` lookups are the obvious candidates for optimization, and I expect there is meaningful room for improvement. This is something I want to revisit, especially if I move forward with `exchange.cpp` where latency will matter more.

What I Would Change / Future Work

There are many things that I still want to work on, and they are all included on the TODO list as unchecked items. I would also like to continue to improve the benchmarking performance and see how I can further optimize my code. The two biggest things that I want to add in terms of functionality are:

1. Different order types, including "fill or kill" and market orders.
2. `exchange.cpp` — A working exchange that allows a user to simulate a stock market with order books for different stocks, the ability to place orders, and see the order books and market information updated live.

I plan to continue working on this order book, with the primary goal of eventually hosting a mock stock exchange on my website.

References

1. https://cppforquants.com/whats-a-trading-order-book/
2. https://www.binance.com/en/academy/articles/understanding-matching-engines-in-trading
3. https://en.wikipedia.org/wiki/Order_matching_system
4. https://www.youtube.com/watch?v=XeLWe0Cx_Lg&t=1989s
5. https://gist.github.com/halfelf/db1ae032dc34278968f8bf31ee999a25
6. https://stackoverflow.com/questions/77116565/how-to-overload-spaceship-operator-with-reve
7. https://stackoverflow.com/questions/12570981/how-to-remove-an-arbitrary-element-from-a-st
8. https://google.github.io/googletest/primer.html